Remote Controller with Fail-Safe Update

Hazard Analysis

Reference	TBD **[TBD-1]**
Issue / revision	Issue 1, Revision 0 — DRAFT
Date	10 October 2026
Author	Andrew Toma
Open items	2 TBD, 4 TBR (App. A)

Document change record

Issue	Rev.	Date	Change
1	0	2026-10-10	First draft

1.0 Purpose and Scope

This document identifies the hazards of the System, the causes of each, its severity, and the controls proposed against it. It is the Phase 0 and Phase 1 delivery of the hazard analysis, combined as for a small project: the top hazards and causes, then all known hazards with their causes and controls [2, Sec. 4].

The controls are proposals; the requirements turn them into rules. The verification of each control is added in the Phase 2 update, once a design exists [2, Sec. 3.1]. This document also decides which software is safety-critical, which the Software Classification left open [6, Sec. 4.0].

The analysis covers the software items of the Software Classification in operation [6, Sec. 3.0]. Bench and build tools are covered where they can affect operation.

2.0 Documents

[1] NASA, *Software Assurance and Software Safety Standard*, NASA-STD-8739.8B, 2022. Causes cite App. A, Table 2, pp. 62–68.
[2] NASA, *NASA Software Engineering Handbook*, NASA-HDBK-2203, Version D, Topic 8.58 “Software Safety and Hazard Analysis”.
[3] NASA, *NASA Software Engineering Handbook*, NASA-HDBK-2203, Version D, Topic 8.05 “SW Failure Modes and Effects Analysis”.
[4] NASA, *Fault Tree Handbook with Aerospace Applications*, Version 1.1, August 2002.
[5] *Remote Controller with Fail-Safe Update — Concept of Operations*, Issue 1, Revision 0, Draft.
[6] *Remote Controller with Fail-Safe Update — Software Classification*, Issue 1, Revision 0, Draft.

3.0 Ground Rules

- **Hazard:** a state, or a set of conditions, with the potential to cause harm [1, p. 13].
- **Sources:** the ConOps and the list of software causes of hazards [1, App. A, Table 2, pp. 62–68]. Numbers of objectives (O), assumptions (A), constraints (C), risks (R), capabilities and scenarios refer to the ConOps [5, Secs. 1.1.2, 3.1, 3.5, 6.0 and 8.0].
- **Points of view:** functional, operational, software, environment, human and organizational [2, Sec. 3.2.2]. The physical view needs the architecture and is added with it.
- **Severity:** the worst credible effect, rated as if no control worked **[TBR-1]**.
- **Credible failures:** up to two at once for a catastrophic hazard, and one for a critical hazard [3, Sec. 3.1] **[TBR-2]**.

- **Likelihood:** not rated yet; it depends on the design [**TBD-2**].
- **Controls:** a control with a scenario number comes from the ConOps. A control without one is new and proposed here.

Legend: severity (names from [3, Sec. 6])

CATASTROPHIC	The controller is lost, or can no longer be commanded, observed or corrected, for the rest of the mission.
CRITICAL	The controller acts against the operator's intent, or cannot be commanded, observed or corrected until the ground acts.
MARGINAL	A function is degraded or interrupted, but none of the above happens.
NEGLIGIBLE	No function is degraded; the operator has extra work or a delay.

Legend: software role [2, Sec. 3.2.2]

CAUSE	A software fault can lead to the hazard.
CONTROL	Software prevents the hazard.
MITIGATION	Software limits the effect once the hazard occurs.
MONITOR	Software detects and reports the hazard.

4.0 Hazard List

The hazards in order of severity [**TBR-3**]:

ID	Hazard	Severity
H-1	The controller is left with no firmware it can run, and cannot be restored	CATASTROPHIC
H-2	Firmware not released by the operator runs on the controller	CATASTROPHIC
H-3	New firmware can no longer reach the controller	CATASTROPHIC
H-4	The controller can no longer be commanded or observed	CATASTROPHIC
H-5	Someone other than the operator commands the System	CATASTROPHIC
H-6	A command is acted on in a form or at a time the operator did not intend	CRITICAL
H-7	Defective firmware released by the operator is confirmed	CRITICAL
H-8	The controller stops working between contacts and is not brought back	CRITICAL
H-9	The ground is shown a false state of the System	CRITICAL
H-10	A fault between contacts never reaches the ground	CRITICAL
H-11	A valid command is not acted on, and the ground is not told	CRITICAL
H-12	A test function acts outside a test	CRITICAL
H-13	A fault response is taken when there is no fault	MARGINAL
H-14	An update cannot be completed	MARGINAL
H-15	Health reports are lost	MARGINAL
H-16	Reports reach the ground later than planned	NEGLIGIBLE
H-17	A command is refused with a wrong or unclear reason	NEGLIGIBLE

5.0 Top-Down Analysis

Figure 1 develops the worst hazard, H-1, from the top down.

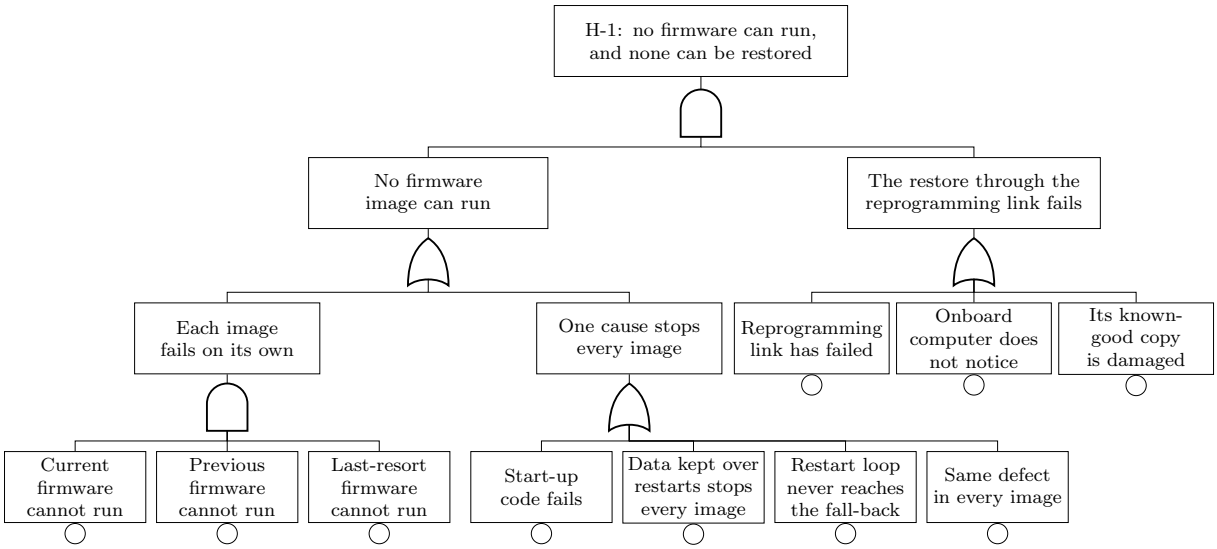


Figure 1: Fault tree of H-1. Symbols as in [4, Table 4-1, p. 34].

Legend: symbols	
Rectangle	An event developed further by the gate below it.
Circle	A basic event, not developed further here.
AND gate	Flat base: the event above occurs if all events below occur.
OR gate	Curved base: the event above occurs if at least one event below occurs.

A minimal cut set is the smallest combination of basic events that causes the top event [4, Sec. 1.3, p. 3]. H-1 has two kinds:

- **Four events:** the current, previous and last-resort firmware all fail on their own, and the restore fails.
- **Two events:** one cause stops every image, and the restore fails.

Two failures at once are credible for a catastrophic hazard (Section 3.0). The causes that stop every image are therefore the weakest point of H-1: they defeat three layers of firmware with one event.

6.0 Bottom-Up Analysis

Before a design exists, the components examined are the System's functions [3, Sec. 4]. Each function is examined for each way it can fail, and each failure is traced to the hazard it leads to.

Function	OMITTED	INCORRECT	UNWANTED	MISTIMED
Telemetry (Capability-1)				
Make health and event reports	H-10, H-15	H-9	H-4, H-15	H-9
Time-tag and store reports	H-10, H-15	H-9	H-15	H-9
Send reports down	H-10, H-15	H-9	H-16	H-16
Keep received reports on the ground	H-10, H-15	H-9	NONE	NONE
Command handling (Capability-2)				
Prepare and check commands on the ground	H-6	H-6	H-6	H-6
Check commands onboard	H-5, H-6	H-6, H-11, H-17	NONE	H-4, H-6, H-11
Hold armed and scheduled commands	H-11	H-6	H-6	H-6
Run commands	H-11	H-6	H-5, H-6	H-6
Answer and log commands	H-11	H-9, H-17	H-9	H-16
Fault handling (Capability-3)				
Detect faults	H-8, H-10	H-10, H-13	H-13	H-8
Respond to faults	H-8	H-1, H-4, H-13	H-13	H-8
Update (Capability-4)				
Prepare and check firmware releases	H-7	H-3, H-7	H-2	NONE
Receive and check firmware	H-1, H-2	H-2, H-3, H-14	H-14	H-2
Install firmware and switch	H-14	H-1	H-6	H-1
Judge and confirm new firmware	H-13	H-3, H-7	H-7	H-7, H-13
Fall back to working firmware	H-1, H-8	H-1, H-2	H-13	H-8, H-13
Restore through the reprogramming link	H-1	H-1, H-2	H-13	H-1
Test (Capability-5)				
Inject faults	NONE	NONE	H-12	H-12

Legend: failure mode

OMITTED	The function is not performed.
INCORRECT	The function is performed with a wrong result.
UNWANTED	The function is performed when it must not be.
MISTIMED	The function is performed too early, too late or out of order.
NONE	The failure leads to no hazard.

Every failure leads to a hazard of the list in Section 4.0, or to none.

7.0 Common Causes

A common cause defeats several controls at once; it can negate a redundancy [4, Sec. 1.3, p. 3]. It includes the same defect in software that runs in more than one place [1, App. A.1.1, p. 62]. The common causes found are:

Wrong onboard clock. Expiry, arm time-outs, scheduled commands, time-tags and the trial time limit all depend on the onboard clock [1, p. 65]. Hazards: H-4, H-6, H-9, H-11, H-13.

Code shared by every image. The start-up code, and any defect common to the current, previous and last-resort firmware, stops all of them at once. Hazards: H-1, H-8.

Data kept over restarts. Stored settings, counters and flags survive a restart. A wrong value can stop every image or keep a wrong state [1, p. 64]. Hazards: H-1, H-6, H-13.

One power supply. The onboard computer and the controller share the spacecraft power supply [5, Sec. 3.3]. A power loss stops the controller and the computer that would restore it, and interrupts reports and commands. Hazards: H-1, H-8, H-10, H-11, H-15.

Same software at both data-link ends. One defect can cut both connections (R-1) [1, p. 66]. Hazards: H-4.

One means of proving origin. If one means proves that both commands and firmware come from the operator, its disclosure opens both. Hazards: H-2, H-5.

Radiation. Bit errors can strike stored firmware, running memory, stored reports, held commands and the arm state (A-5) [1, p. 63]. Hazards: H-1, H-6, H-8, H-9, H-10.

One set of build tools. The same tools build every image; a tool defect enters all of them. Hazards: H-1, H-7, H-12.

Onboard computer software that cannot be updated. It stays fixed for the mission [5, Sec. 5.0], so a defect in it is never corrected. Hazards: H-1, H-3, H-4.

One engineer and AI-drafted code. The same misunderstanding can enter a function and the check meant to catch it (C-3, R-2) [1, p. 67]. Hazards: all.

8.0 Hazard Reports

Each report gives the conditions, causes, worst effect, proposed controls, how the controls can fail, and the common causes that defeat them [2, Sec. 4].

H-1 The controller is left with no firmware it can run, and cannot be restored.
Severity: CATASTROPHIC. Software: CAUSE, CONTROL.

When During an update, at any restart, or when a restore is needed.

Causes
- (a) Power is lost while firmware is written (Scenario-600) [1, p. 63].
- (b) A bit error changes stored firmware (Scenario-610) [1, p. 63].
- (c) An update writes over the firmware the controller would fall back to [1, p. 65].
- (d) The new and the previous firmware both fail (Scenario-460).
- (e) The last-resort firmware has changed unnoticed by the time it is needed [1, p. 63].
- (f) A restart loop never reaches the fall-back [1, p. 64].
- (g) The start-up code fails, or its stored copy is damaged [1, p. 63].
- (h) The reprogramming link has failed, or the onboard computer does not notice that a restore is needed (Scenario-520).
- (i) The onboard computer's known-good copy of the firmware is damaged [1, p. 63].

Effect The controller is lost for the rest of the mission; nobody can reach it (A-1).

Controls
1. An update never writes over the firmware the controller would fall back to (Scenario-600).
2. Every firmware image is checked before it runs (Scenario-610).
3. Failed starts are counted; after a set number, the controller returns to its previous firmware (Scenario-440).
4. A last-resort firmware that updates never replace (Scenario-460).
5. A restore through the reprogramming link, which does not depend on the controller's software (Scenario-480).
6. Stored firmware and the reprogramming link are checked between uses, so a hidden fault is reported before it is needed.

Control fails if The image check passes damaged firmware; the failed-start count is lost at each restart; the last-resort firmware shares the defect that stops the others.

Common causes Code shared by every image; data kept over restarts; one power supply; radiation; one set of build tools; onboard computer software that cannot be updated.

H-2 Firmware not released by the operator runs on the controller.
Severity: CATASTROPHIC. Software: CONTROL.

When During an update, a fall-back or a restore.

Causes
(a) Firmware not from the operator arrives (Scenario-410) [1, p. 67].
(b) Firmware is changed after it is checked, in transfer or in storage [1, p. 63].
(c) Firmware older than the minimum version, which the operator no longer releases, is sent again (Scenario-420).
(d) The means of proving that firmware comes from the operator is disclosed.
(e) A fall-back or a restore starts an image that was never released.

Effect Firmware of unknown behaviour runs the controller. It can refuse further updates and report a false state. The restore is used only when no firmware can run (Scenario-480), so firmware that runs is never removed.

Controls
1. The onboard computer refuses firmware that is not released by the operator (Scenario-410), damaged (Scenario-400), older than the minimum version (Scenario-420), or built for another controller (Scenario-360).
2. The controller checks firmware again before it runs it (Scenario-610).
3. The operator can order a restore through the reprogramming link while firmware is running.

Control fails if The check accepts what it should refuse; the check is made before the last transfer, not after it.

Common causes One means of proving origin; radiation.

H-3 New firmware can no longer reach the controller.
Severity: CATASTROPHIC. Software: CAUSE, CONTROL.

When After an update is confirmed, or after a fault in the onboard computer.

Causes
(a) Confirmed firmware runs, but cannot receive new firmware.
(b) A defect in the onboard computer's update path.
(c) A stored minimum version or controller identity is wrong, so all new firmware is refused [1, p. 67].
(d) Storage for firmware on the onboard computer stays full [1, p. 66].

Effect The controller can no longer be corrected for the rest of the mission.

Controls
1. New firmware runs a trial before it is confirmed (Scenario-430 to Scenario-450).
2. New firmware is judged good only after it has received firmware itself.
3. The last-resort firmware can always receive new firmware.
4. The operator can order a return to the last-resort firmware.

Control fails if The trial does not test the update path; the defect is in the onboard computer, which no control can reach.

Common causes Onboard computer software that cannot be updated; data kept over restarts.

H-4 The controller can no longer be commanded or observed.
Severity: CATASTROPHIC. Software: CAUSE, CONTROL.

When At any time.

Causes
(a) Both data-link connections fail (Scenario-510).
(b) One software defect cuts both connections (R-1) [1, p. 66].
(c) A controller that never stops sending blocks the data link [1, p. 63].
(d) The onboard clock is wrong, so every command is refused as expired, including the one that would correct the clock [1, p. 65].
(e) The ground loses the means of proving that commands come from the operator, so no command is accepted.

(f) A fault response marks both connections failed (H-13).

Effect The controller runs on with its current firmware, but nobody can command or observe it for the rest of the mission.

Controls
1. Two separate data-link connections; one is enough (O-3, Scenario-500).
2. The loss of a connection is reported (Scenario-500, Scenario-510).
3. The reprogramming link stays usable (Scenario-510).
4. The command that corrects the onboard clock does not depend on the onboard clock.

Control fails if Both connections fail from one cause. No control yet covers a lost means of proving origin.

Common causes Same software at both data-link ends; wrong onboard clock; onboard computer software that cannot be updated.

H-5 Someone other than the operator commands the System.
Severity: CATASTROPHIC. Software: CONTROL.

When During a contact, or at any time for a source on the spacecraft.

Causes
(a) A command not from the operator arrives (Scenario-300) [1, p. 67].
(b) The means of proving that a command comes from the operator is disclosed.
(c) Someone other than the operator uses the operator interface; no second person checks a command (A-4).
(d) A command arrives from an unexpected system on the spacecraft [1, p. 62].

Effect Another party can do anything the operator can, and hide it. Control of the controller is lost for the rest of the mission.

Controls
1. Every command is proven to come from the operator; any other is refused and logged (O-2, Scenario-300).
2. Only the operator can use the operator interface.

Control fails if The means of proof is disclosed; the check accepts what it should refuse.

Common causes One means of proving origin.

H-6 A command is acted on in a form or at a time the operator did not intend.
Severity: CRITICAL. Software: CAUSE, CONTROL.

When While a command is checked, held or run.

Causes
(a) A command is changed in transfer [1, p. 62].
(b) An old command is sent again (Scenario-300).
(c) A command is run twice after a lost answer (Scenario-350).
(d) A command not allowed in the current mode arrives (Scenario-310) [1, p. 63].
(e) A command with impossible values arrives (Scenario-320) [1, p. 62].
(f) A dangerous command arrives without a valid arm (Scenario-330, Scenario-335) [1, p. 63].
(g) A command arrives after its expiry (Scenario-340) [1, p. 63].
(h) A scheduled command runs at the wrong time, or again after a restart [1, p. 64].
(i) A bit error changes a held command or the arm state [1, p. 63].
(j) The ground software builds a wrong command, or the operator enters one (A-4) [1, p. 63].

Effect The controller does what the operator did not intend, and the ground learns of it only at the next contact.

Controls
1. The onboard computer checks that every command is new, allowed in the current mode, has possible values and has not expired (Scenario-120).

2. A dangerous command needs an arm, which expires (Scenario-330, Scenario-335).
3. A command sent again is not run twice; the answer is "already done" (Scenario-350).
4. The ground software checks every command before it is sent (Scenario-120).
5. Every command and its answers are logged (Scenario-120).

Control fails if The checks use a wrong onboard clock; a held command or the arm state changes after it is checked.

Common causes Wrong onboard clock; radiation; data kept over restarts.

H-7 Defective firmware released by the operator is confirmed.
Severity: CRITICAL. Software: CAUSE, CONTROL.

When During the trial of new firmware, and after it is confirmed.

Causes
(a) The trial does not exercise the defect.
(b) The wrong build is released [1, p. 67].
(c) The release carries wrong configuration values [1, p. 67].
(d) Firmware built for a different controller is released (Scenario-360).
(e) The build tools put a defect into the firmware.
(f) Code is released without being fully tested [1, p. 67].

Effect The controller misbehaves until a further update. If the defect stops updates, H-3 follows.

Controls
1. New firmware not judged good within the trial time is given up (Scenario-430 to Scenario-450).
2. Firmware built for another controller is refused (Scenario-360).
3. The ground software checks every release before it is sent (Scenario-100).
4. The operator can order a return to the previous firmware.

Control fails if The judgement checks too little; the trial is shorter than the time the defect takes to show.

Common causes One set of build tools; one engineer and AI-drafted code.

H-8 The controller stops working between contacts and is not brought back.
Severity: CRITICAL. Software: CAUSE, CONTROL, MONITOR.

When In any controller mode.

Causes
(a) A bit error in the memory the controller runs from (Scenario-620) [1, p. 63].
(b) The firmware hangs, deadlocks or overloads [1, p. 63].
(c) The controller restarts but does not resume [1, p. 63].
(d) The restart takes longer than the time to the next failure [1, p. 64].
(e) The controller misbehaves when no command arrives for a long time [1, p. 65].
(f) The onboard computer does not notice the silence, or its watching program has crashed (Scenario-520, Scenario-530).

Effect The controller's work stops until the ground acts at the next contact.

Controls
1. The controller restarts on its own and resumes (Scenario-620).
2. The onboard computer notices the silence, tries to bring the controller back, and records what it did (Scenario-520).
3. A program on the onboard computer that crashes restarts and resumes (Scenario-530).
4. New firmware that hangs is given up (Scenario-430).

Control fails if The controller's own restart and the onboard computer's watch fail from one fault.

Common causes Radiation; data kept over restarts; one power supply; code shared by every image.

H-9 The ground is shown a false state of the System.
Severity: CRITICAL. Software: CAUSE, CONTROL.

When At every contact.

Causes
(a) Reports carry a wrong time-tag [1, p. 64].
(b) Old reports are shown as current [1, p. 63].
(c) A report is changed in storage or in transfer [1, p. 62].
(d) An answer says "done" or "already done" for a command that did not run.
(e) The ground software stores or shows a report wrongly [1, p. 62].

Effect The operator decides on false information: a harmful command can follow (H-6), or a fault is missed (H-10).

Controls
1. The onboard computer time-tags every report (Scenario-110).
2. Every report is checked for damage when it arrives.
3. The ground software keeps every report received [5, Sec. 3.2].

Control fails if The onboard clock is wrong; a report is damaged before its check is made.

Common causes Wrong onboard clock; radiation.

H-10 A fault between contacts never reaches the ground.
Severity: CRITICAL. Software: CAUSE, MONITOR.

When Between contacts.

Causes
(a) The fault is not detected [1, p. 66].
(b) It is detected, but no event report is made [1, p. 65].
(c) The event report is lost: storage is full, a program crashes, or power is lost before it is stored (Scenario-210, Scenario-530).
(d) The ground loses it after it arrives.

Effect The ground does not know of the fault. Hidden faults pile up, for example a damaged last-resort firmware (H-1).

Controls
1. Event reports are kept when storage is full (Scenario-210).
2. No report is lost without the ground being told (O-5).
3. Stored reports survive a power loss and a restart.

Control fails if Fault detection depends on the part that has failed.

Common causes One power supply; radiation.

H-11 A valid command is not acted on, and the ground is not told.
Severity: CRITICAL. Software: CAUSE, CONTROL.

When While a command is checked, held or run.

Causes
(a) The command is lost in a full buffer [1, p. 63].
(b) It is refused as expired because the onboard clock is wrong (Scenario-340).
(c) A scheduled command is lost at a restart (Scenario-130).
(d) Heavy traffic on the data link stops it getting through [1, p. 63].
(e) Its answer is lost (Scenario-350).

Effect The operator believes an action was taken that was not; for example, a cancelled update goes on (Scenario-470).

Controls
1. Every command gets two answers: received, then done with the result (Scenario-120).
2. A refused command is answered with the reason (Scenario-310 to Scenario-340).
3. Scheduled commands survive a restart.

Control fails if The answers are lost along with the command.

Common causes Wrong onboard clock; one power supply.

H-12 A test function acts outside a test.
Severity: CRITICAL. Software: CAUSE.

When In operation.
Causes (a) Fault-injection functions stay in the operating software [1, p. 64].
(b) A wrong command, or one not from the operator, starts them.
Effect Power is cut, a data-link connection is broken, or the radio link is degraded during operation. Each is a cause of H-1 or H-4.
Controls 1. Test functions are for verification only (Capability-5).
2. Test functions are not part of the operating software.
Control fails if A bench build is released for operation [1, p. 67].
Common causes One set of build tools.

H-13 A fault response is taken when there is no fault.
Severity: MARGINAL. Software: CAUSE.

When In any controller mode.
Causes (a) Fault detection reports a fault that is not there [1, p. 66].
(b) The trial time is too short for good firmware to be judged good (Scenario-450).
(c) A failed-start count or a flag is not cleared [1, p. 66].
Effect Good firmware is given up, the controller restarts for no reason, or a good data-link connection is marked failed. Repeated, it becomes a restart loop (H-1) or the loss of both connections (H-4).
Controls 1. Every fault response is recorded and reported (Capability-3).
2. A given-up update can be repeated (Scenario-600).
Control fails if The record of the response is lost with it.
Common causes Data kept over restarts; wrong onboard clock.

H-14 An update cannot be completed.
Severity: MARGINAL. Software: CAUSE, CONTROL.

When During an update.
Causes (a) The radio link is lost again and again during the upload (Scenario-200).
(b) A resumed upload is put together wrongly and refused.
(c) Storage on the onboard computer is full of reports (Scenario-210) [1, p. 66].
(d) Good firmware is refused because the check is wrong.
Effect The current firmware keeps running; the correction waits. If it never completes, H-3 follows.
Controls 1. An upload resumes from where it stopped (Scenario-200).
2. A crashed program on the onboard computer resumes the update (Scenario-530).
3. The operator can cancel an update (Scenario-470).
Control fails if The point where the upload stopped is stored wrongly.
Common causes Data kept over restarts.

H-15 Health reports are lost.
Severity: MARGINAL. Software: CAUSE, MITIGATION.

When	Between contacts.
Causes	(a) Storage fills during a long gap between contacts (Scenario-210). (b) Power is lost, or a program crashes, before a report is stored. (c) A bit error damages stored reports [1, p. 62].
Effect	The health history between contacts has gaps; a slow change can be missed. Event reports are covered by H-10.
Controls	1. The oldest health reports are dropped first, and the ground is told how many and over what time span (Scenario-210).
Control fails if	The count of dropped reports is itself lost.
Common causes	One power supply; radiation.

H-16 Reports reach the ground later than planned.
Severity: NEGLIGIBLE. Software: CAUSE.

When	At a contact after a long gap.
Causes	(a) A contact is too short to send everything stored (A-3).
Effect	No report is lost; the operator sees some of them a contact later.
Controls	1. Event reports are sent before health reports.
Control fails if	Event reports alone fill the contact.
Common causes	None.

H-17 A command is refused with a wrong or unclear reason.
Severity: NEGLIGIBLE. Software: CAUSE.

When	When a command is refused.
Causes	(a) The ground and the onboard software use different lists of commands or reasons [1, p. 66]. (b) Errors are handled differently in different places [1, p. 63].
Effect	The operator spends a contact finding out why, then sends the command again.
Controls	1. One list of commands and refusal reasons, shared by the ground and the onboard software.
Control fails if	The two copies of the list drift apart.
Common causes	None.

9.0 Must-Work and Must-Not-Work Functions

The hazard reports identify the functions that must work and those that must not work [1, App. A.1.2, p. 62].

Must work

- Return to working firmware when new firmware fails (H-1).
- Restore the controller through the reprogramming link (H-1, H-2).
- Check every firmware image before it runs (H-1, H-2).
- Refuse commands and firmware not from the operator (H-2, H-5).
- Accept the command that corrects the onboard clock (H-4).
- Detect and report faults between contacts (H-8, H-10).
- Keep and send down every event report (H-10).
- Answer every command (H-11).

Must not work

- Write over the firmware the controller would fall back to, or over the last-resort firmware (H-1).

- Run firmware not released by the operator (H-2).
- Run a command not from the operator (H-5).
- Run a dangerous command without a valid arm (H-6).
- Run a command twice (H-6).
- Confirm firmware that has not been judged good (H-7).
- Inject faults outside a test (H-12).

10.0 Safety-Critical Software

Software is safety-critical if a hazard analysis finds that it meets at least one of the criteria a to e [1, Sec. 4.2, p. 20]. Until the architecture divides each item into components, the whole item is treated as safety-critical [**TBR-4**].

Software item	Safety-critical	Criteria	Hazards
Controller firmware	YES	a, b, c, e	H-1 to H-4, H-6 to H-13
Onboard computer software	YES	a, b, c, e	H-1 to H-17
Ground software	YES	a, b	H-2, H-4 to H-7, H-9, H-10, H-15, H-17
Bench test tools	NO	NONE	NONE
Firmware build tools	NO	NONE	NONE

Legend: criteria [1, Sec. 4.2, p. 20]

a Causes or contributes to a system hazardous condition or event.
b Controls functions identified in a system hazard.
c Provides mitigation for a system hazardous condition or event.
d Mitigates damage if a hazardous condition or event occurs.
e Detects, reports and takes corrective action if the system reaches a potentially hazardous state.

Note. *Bench test tools stay out of operation (H-12). Firmware build tools never run in operation; a defect they put into firmware is a common cause (Section 7.0).*

A Open Items

The values below are not yet known.

TBD-1 Project name and document reference.
TBD-2 Likelihood and risk of each hazard, rated once the architecture exists.

The proposals below await the author's acceptance.

TBR-1 Severity rated as the worst credible effect with no control working, and the definitions of the four severity categories.
TBR-2 Up to two failures at once credible for a catastrophic hazard, and one for a critical hazard.
TBR-3 The hazard list and the severity of each hazard.
TBR-4 The safety-critical designation of each software item.